1. Overview

This project implements a **Decentralized Auction System** using Solidity.
The contract allows users to:

- Create auctions
- Place bids
- Withdraw funds
- Finalize auctions

Gas optimisation was applied to reduce transaction costs and improve execution efficiency.

2. Key Optimizations Applied

- **Struct Packing (MAJOR Improvement):**
  Before:
  uint256 highestBid;
  uint256 startingPrice;
  uint256 deadline;
  bool ended;

  After:
  uint96 highestBid;
  uint96 startingPrice;
  uint64 deadline;
  bool ended;

Why this improves gas:

- Solidity stores variables in **32-byte** storage slots
- Smaller data types allow multiple variables to fit in one slot **(storage packing)**
- This reduces:
  - Storage writes (SSTORE)
  - Storage reads (SLOAD)

Result: Significant gas reduction during createAuction() and updates

3. Use of calldata instead of memory

Applied in:

function **createAuction**(
  string calldata _itemName,
  string calldata _metadataCID

Why this improves gas:

- calldata is read-only and cheaper

- Avoids unnecessary memory allocation

Result: Lower gas cost for external function calls

4. Caching Storage Variables

Example:

AuctionItem storage a = auctions[_auctionId];

Why this improves gas:

- Avoids repeated access like:

auctions[_auctionId]. highestBid

- Each storage read is expensive

Result: Reduces repeated **SLOAD** operations

5. Pull Over Push Payment Pattern

Instead of:

- Directly sending ETH inside loops or functions

Used:

pendingReturns[_auctionId][prevBidder] += prevBid;

Why this improves gas:

- Avoids failed transactions due to transfer issues
- Reduces unnecessary external calls

Result:

- More efficient and safer execution

6. Increment Optimization

Before:

auctionCount++;

After:

uint256 id = ++auctionCount;

Why this improves gas:

- Slightly cheaper due to pre-increment usage
- Avoids extra temporary operations

Minor but measurable improvement

7. Reduced Redundant Storage Access

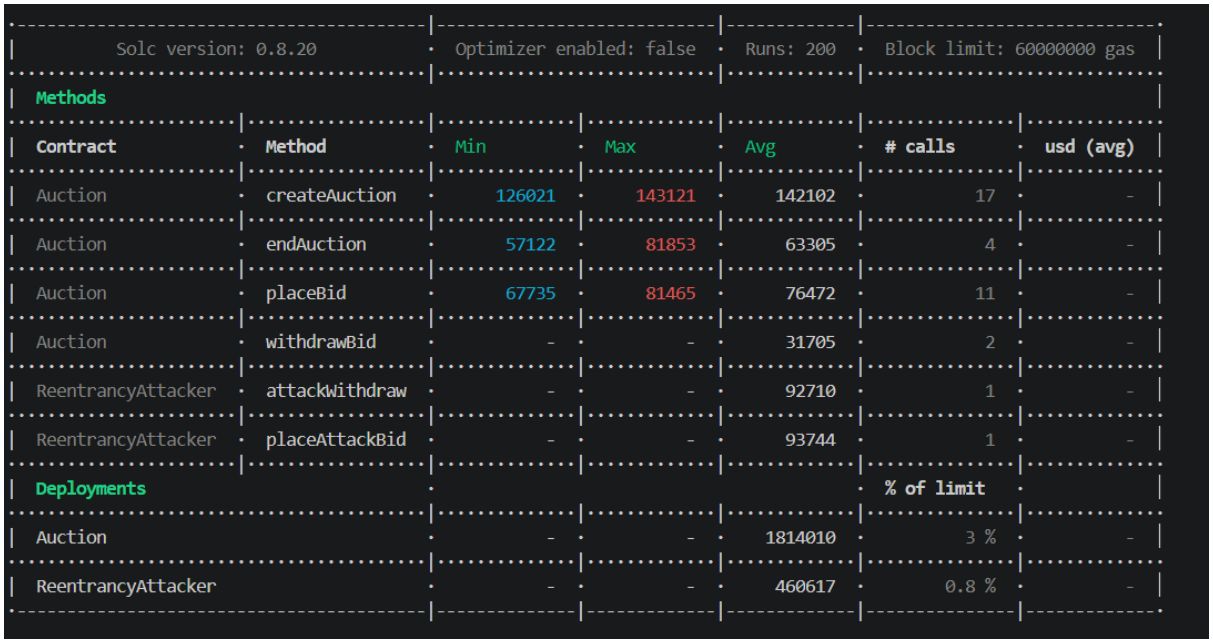
Using:

address prevBidder = a.highestBidder;
uint256 prevBid = a.highestBid;

Why this improves gas:

- Prevents multiple reads from storage
- Uses stack variables instead

Before:



Solc version: 0.8.20	Optimizer enabled: false	Runs: 200	Block limit: 60000000 gas

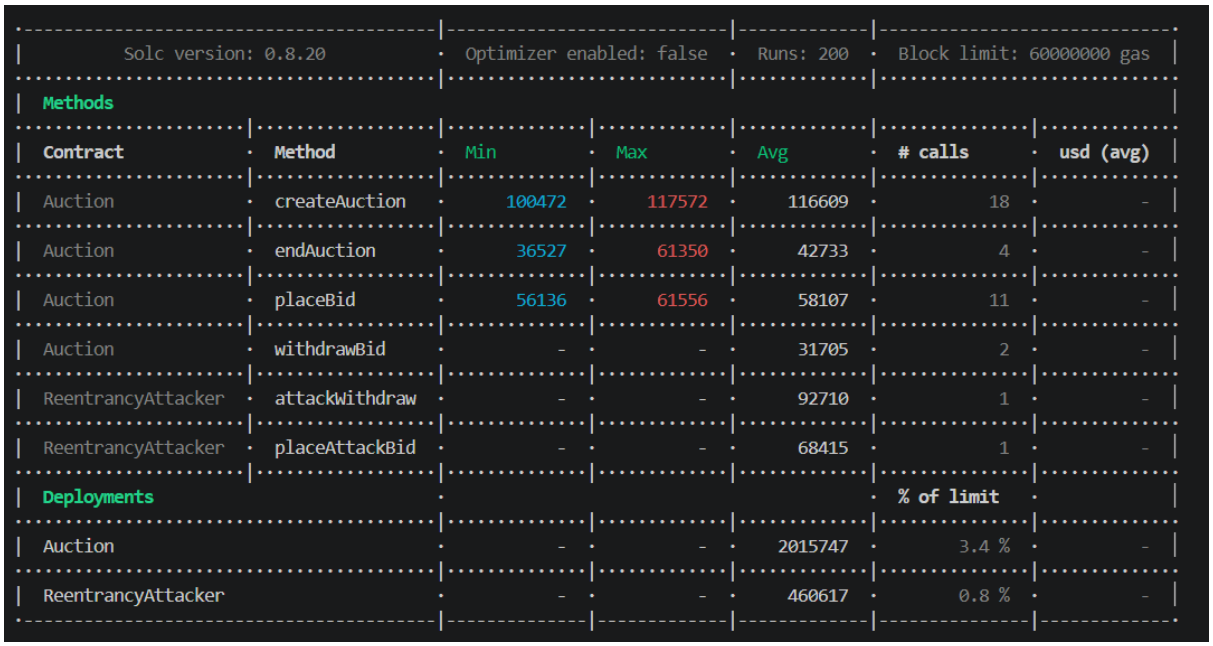
Methods

Contract	Method	Min	Max	Avg	# calls	usd (avg)
Auction	createAuction	126021	143121	142102	17	-
Auction	endAuction	57122	81853	63305	4	-
Auction	placeBid	67735	81465	76472	11	-
Auction	withdrawBid	-	-	31705	2	-
ReentrancyAttacker	attackWithdraw	-	-	92710	1	-
ReentrancyAttacker	placeAttackBid	-	-	93744	1	-

Deployments

			% of limit	
Auction	-	-	1814010	3 %	-
ReentrancyAttacker	-	-	460617	0.8 %	-

After:



Solc version: 0.8.20	Optimizer enabled: false	Runs: 200	Block limit: 60000000 gas

Methods

Contract	Method	Min	Max	Avg	# calls	usd (avg)
Auction	createAuction	100472	117572	116609	18	-
Auction	endAuction	36527	61350	42733	4	-
Auction	placeBid	56136	61556	58107	11	-
Auction	withdrawBid	-	-	31705	2	-
ReentrancyAttacker	attackWithdraw	-	-	92710	1	-
ReentrancyAttacker	placeAttackBid	-	-	68415	1	-

Deployments

			% of limit	
Auction	-	-	2015747	3.4 %	-
ReentrancyAttacker	-	-	460617	0.8 %	-